SR UNIVERSITY

AI Assistant Coding
Assignment-19.3

Name of Student : V. Abhinav Batch : 41
Enrollment No. : 2303A52486

Task 1: Python to C++ Conversion

Translate a simple Python class into C++ using AI assistance.

Instructions:

- **Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code**
- **Ensure proper handling of:**
  - **Constructors**
  - **Data types**
  - **Access specifiers**
- **Compile and run both versions to verify output consistency**

Prompt:

Convert a Python class Student with attributes name, age and a display() method into equivalent C++ code with constructor and proper access specifiers.

Python Code:

```
class Student:

    def __init__(self, name, age):

        self.name = name

        self.age = age



    def display(self):

        print(self.name, self.age)
```

```
s = Student("John", 20)

s.display()
```

C++ Code:

```
#include <iostream>

using namespace std;


class Student {

private:

   string name;

   int age;


public:

   Student(string n, int a) {

       name = n;

       age = a;

   }


   void display() {

       cout << name << " " << age << endl;

   }

};
```

```
int main() {

    Student s("John", 20);

    s.display();

    return 0;
```

Output:

```
/student_demo.cpp -o /tmp/student_demo
/tmp/student_demo
John 20
```

```
/student_demo.cpp -o /tmp/student_demo && /tmp/student_demo
John 20
```

Explanation:

Constructor added explicitly in C++
Data types defined (string, int)
Encapsulation via private/public
Output verified same in both languages

Task 2: Java to Python Function Conversion

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- **Ask AI to translate a Java isPrime() method into Python**
- **Test the function with multiple inputs**
- **Ensure the Python version follows Pythonic syntax and logic**

Prompt:

Convert Java method isPrime(int n) into Python function using Pythonic syntax.

Java Code:

```
public static boolean isPrime(int n) {
   if(n <= 1) return false;
   for(int i=2; i<=n/2; i++) {
       if(n % i == 0) return false;
   }
   return true;
}
```

Python Code:

```
def is_prime(n):
   if n <= 1:
       return False
   for i in range(2, n//2 + 1):
       if n % i == 0:
           return False
   return True


print(is_prime(7))
print(is_prime(10))
```

Output:

```
tcoding/pythonpractice$ /usr/bin/python3 /home/shashidhar/Downloads/aiassitantcoding-20260
312T084629Z-3-001/aiassitantcoding/pythonpractice/prime.py
True
False
```

Explanation:

Python uses indentation instead of braces
range() replaces loop structure
Cleaner syntax (no type declaration)
Logic preserved exactly

Task 3: Pseudocode to Python Implementation

Translate a given pseudocode algorithm into Python using AI.

Instructions:

1. **Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)**
2. **Ask AI to convert it into executable Python code**
3. **Validate correctness using sample input lists**

Prompt:

Convert bubble sort pseudocode into Python program.

Code:

```
def bubble_sort(arr):

    n = len(arr)

    for i in range(n):

        for j in range(0, n-i-1):
```

```
            if arr[j] > arr[j+1]:

                arr[j], arr[j+1] = arr[j+1], arr[j]

    return arr



data = [5, 2, 9, 1]

print(bubble_sort(data))
```

Output:

```
tcoding/pythonpractice$ /usr/bin/python3 /home/shashidhar/Downloads/aiassitantcoding-20260
312T084629Z-3-001/aiassitantcoding/pythonpractice/bubble.py
[1, 2, 5, 9]
```

Explanation:

Pseudocode loops directly mapped to Python loops
Swap done using tuple unpacking
Algorithm correctness verified

Task 4: SQL to Pandas Query

Translate a SQL query into a Pandas DataFrame operation in

Python.

Instructions:

- **Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).**
- **Ask AI to generate equivalent Pandas code.**
- **Test the generated code on a sample DataFrame**

Prompt:

Convert SQL query SELECT name, salary FROM employees WHERE salary > 50000 into Pandas code.

Code:

SQL Query:

```
SELECT name, salary

FROM employees

WHERE salary > 50000;

import pandas as pd



data = {

   "name": ["A", "B", "C"],

   "salary": [40000, 60000, 70000]

}

df = pd.DataFrame(data)

result = df[df["salary"] > 50000][["name", "salary"]]

print(result)
```

Output:

```
/.venv/bin/python /home/shashidhar/Downloads/aiassitantcoding-20260312T084629Z-3-001/aiass
itantcoding/pandas_demo.py
  name  salary
1    B   60000
2    C   70000
```

Explanation:

WHERE → Boolean filtering
SELECT → column selection
Equivalent data subset returned

Task 5: Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming

languages.

Examples:

- **Converting a Python searching algorithm into C**
- **Translating a Java sorting program into JavaScript**

Instructions:

- **Use AI to translate a selected algorithm between two programming languages**
- **Execute and test both versions**
- **Document translation challenges such as:**
  - **Syntax differences**
  - **Library support**
  - **Memory management**

Prompt:

Convert Python linear search algorithm into C language with proper memory handling.

Code:

```
def search(arr, key):

   for i in range(len(arr)):

       if arr[i] == key:

           return i
```

```
    return -1

print(search([1,2,3,4], 3))
```

C Code:

```
#include <stdio.h>

int search(int arr[], int n, int key) {

    for(int i = 0; i < n; i++) {

        if(arr[i] == key)

            return i;

    }

    return -1;

}

int main() {

    int arr[] = {1,2,3,4};

    int result = search(arr, 4, 3);

    printf("%d\n", result);

    return 0;

}
```

Output:

Explanation:

Manual array handling in C
No dynamic typing → explicit sizes required
Memory handled via static arrays
Same logic preserved